ENGR 102 Summer Bridge
Day 7 Instructor Key

Ordered Validation, Priority, and Nested Decisions

20 points • approximately 20-25 minutes

Name		UIN		Section	

Boolean-operator convention. Python operators appear as `and`, `or`, and `not`. Their lowercase spelling is required in code.

Show intermediate state for trace credit. Program credit requires one coherent solution. Unless a prompt says otherwise, give exact Python 3 output.

Question 1. Trace a priority-sensitive validation chain. 5 points

```
quantity = -2

if quantity < 0:
    message = "invalid"
elif quantity <= 10:
    message = "small"
elif quantity <= 25:
    message = "medium"
else:
    message = "large"

print(message)
```

Name the first matching branch and explain why a negative value cannot reach small.

Answer and explanation

quantity < 0 is True, so invalid is assigned and the rest of the chain is skipped. A single if/elif chain preserves priority; exact output is invalid.

Question 2. Trace separate cost and classification decisions. 5 points

```
mass = 28
purity = 90
rush = "yes"
cost = 12 + 1.8 * mass

if rush == "yes":
    cost = cost + 6

if purity < 85 or mass > 25:
    label = "hold"
else:
    label = "accepted"

print(label, cost)
```

Show cost state before and after rush, then evaluate both classification components and give exact output.

Answer and explanation

Base cost is 12 + 1.8 * 28 = 62.4; rush raises it to 68.4. purity < 85 is False, but mass > 25 is True, so the `or` condition assigns hold. Exact output: hold 68.4.

Question 3. Program construction — 10 points

Write one complete Python program that satisfies every requirement.

- Read score as an integer and missing as text.
- Print invalid when score is outside 0 through 100.
- For valid input, print incomplete when missing is exactly yes, regardless of score.
- Otherwise print distinction for score at least 85, pass for score at least 70, and review for every lower valid score.
- Print exactly Result: followed by the label.

Answer and explanation

```
score = int(input())
missing = input()

if score < 0 or score > 100:
    print("invalid")
elif missing == "yes":
    print("Result: incomplete")
elif score >= 85:
    print("Result: distinction")
elif score >= 70:
    print("Result: pass")
else:
    print("Result: review")
```

Invalid data has first priority. Among valid records, missing work has priority over the numeric labels, so a high score cannot bypass incomplete.

Protective tests include 101/no -> invalid, 88/yes -> incomplete, 88/no -> distinction, 70/no -> pass, and 69/no -> review.

Scoring guidance

2 input/domain, 2 missing priority, 3 descending score branches, 2 exact output and mutual exclusion, 1 boundary tests.